

# Symposium RAG Bot - Complete Beginner's Guide

---

## Table of Contents

1. [What is This Project?](#)
  2. [What is RAG?](#)
  3. [Project Architecture](#)
  4. [Understanding the Dependencies](#)
  5. [Setup and Installation](#)
  6. [Phase 1: Data Ingestion \(The Preparation Phase\)](#)
  7. [Phase 2: Backend API \(The Brain\)](#)
  8. [Phase 2: Frontend UI \(The Interface\)](#)
  9. [Phase 3: Semantic Caching with BetterDB](#)
  10. [How Everything Works Together](#)
  11. [Key Concepts Explained](#)
- 

## What is This Project?

This is a **Retrieval-Augmented Generation (RAG) chatbot** that can answer questions about a specific PDF document (in this case, a symposium document).

Think of it like a smart assistant that has read a document and can answer questions about it by:

- Finding relevant sections in the document
  - Using those sections to generate accurate answers
  - Showing you where the information came from
- 

## What is RAG?

**RAG stands for Retrieval-Augmented Generation.**

Let's break this down:

### The Problem RAG Solves

Imagine you have a large document and you want to ask questions about it. You could:

1. Give the entire document to an AI (like ChatGPT) every time you ask a question
2. But that's expensive, slow, and sometimes the document is too large!

### The RAG Solution

RAG solves this by:

1. **Breaking down** the document into smaller chunks
2. **Storing** these chunks in a special database (vector store)
3. **Finding** only the relevant chunks when you ask a question

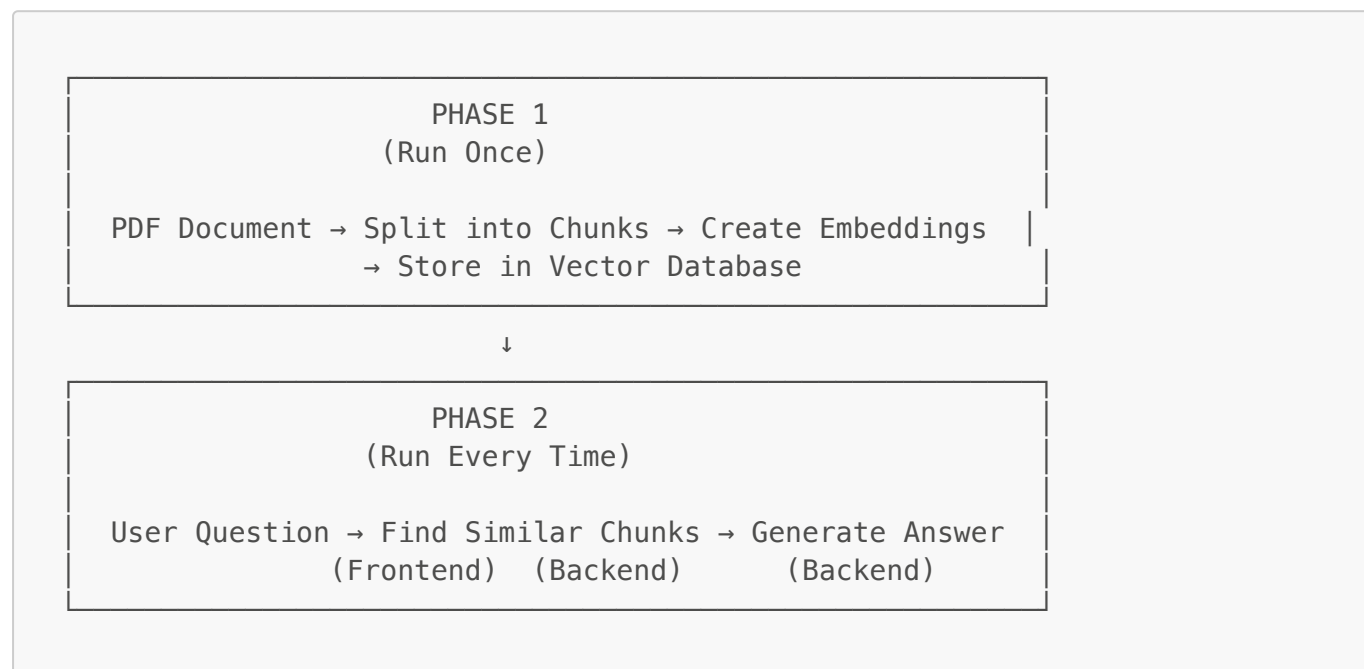
#### 4. **Giving** only those relevant chunks to the AI to generate an answer

**Analogy:** Think of it like a librarian who knows where every book is. Instead of reading the entire library to answer your question, they quickly find the 2-3 relevant books and use only those to help you.

---

## Project Architecture

This project has two main phases:



## File Structure

```
Symposium_Bot/
├── data/
│   └── symposium.pdf           # Your source document
├── vector_store/
│   ├── index.faiss             # Created after Phase 1
│   ├── index.pkl               # The vector database
│   └── metadata.pkl            # Metadata
├── phase1_ingestion/
│   └── ingest.py                # Prepares the data (Run ONCE)
├── phase2_retrieval/
│   ├── backend.py              # API server (FastAPI)
│   └── frontend.py             # Chat interface (Streamlit)
├── .env                        # Your OpenAI API key
├── requirements.txt            # Python packages needed
└── README.md                   # Quick start guide
```

## Understanding the Dependencies

Let's understand what each package in [requirements.txt](#) does:

```
langchain==0.3.15                # Framework for building LLM
applications                      #
langchain-openai==0.2.14         # OpenAI integrations for LangChain
langchain-community==0.3.15     # Community tools (PDF loader, FAISS)
faiss-cpu==1.13.0               # Vector database for similarity
search                          #
pypdf==5.1.0                    # Reads PDF files
fastapi==0.115.6                # Web framework for the backend API
uvicorn==0.34.0                 # Server to run FastAPI
streamlit==1.41.1               # Framework for the chat interface
python-dotenv==1.0.1            # Loads environment variables from
.env                             #
requests==2.32.3                # Makes HTTP requests to the backend
numpy                           # Numerical computing library
betterdb-semantic-cache         # Semantic cache backed by Valkey
vector search
```

## Key Terms:

- **LangChain:** A framework that makes it easier to build applications with Large Language Models (LLMs)
- **FAISS:** Facebook AI Similarity Search - a library for fast similarity search
- **FastAPI:** A modern Python web framework for building APIs
- **Streamlit:** A Python library for creating web interfaces quickly
- **BetterDB Semantic Cache:** A caching library that uses vector similarity to cache LLM responses — catches not just exact repeat questions but also paraphrases
- **Valkey:** An open-source, high-performance key-value store (Redis fork backed by AWS, Google, and Oracle). Used here as the storage backend for the semantic cache

---

## Setup and Installation

Before running the project, you need to set up your Python environment and install all required dependencies.

### Prerequisites

- **Python 3.8 or higher** installed on your system
- **OpenAI API Key** (get one from <https://platform.openai.com/api-keys>)
- **Terminal/Command Prompt** access

---

### Step 1: Create a Virtual Environment

A virtual environment isolates your project dependencies from other Python projects on your system.

#### For Mac/Linux:

```
# Navigate to the project directory
cd /path/to/Symposium_Bot

# Create a virtual environment named .venv
python3 -m venv .venv

# Activate the virtual environment
source .venv/bin/activate
```

### For Windows:

```
# Navigate to the project directory
cd C:\path\to\Symposium_Bot

# Create a virtual environment named .venv
python -m venv .venv

# Activate the virtual environment
.venv\Scripts\activate
```

**How to know it's activated:** You should see `(.venv)` at the beginning of your command prompt:

```
(.venv) user@computer:~/Symposium_Bot$
```

---

## Step 2: Install Dependencies

With the virtual environment activated, install all required packages:

```
pip install -r requirements.txt
```

### What this does:

- Reads the [requirements.txt](#) file
- Downloads and installs all listed packages
- Ensures you have the exact versions needed

### Expected output:

```
Collecting langchain==0.3.15
  Downloading langchain-0.3.15-py3-none-any.whl
```

```
...  
Successfully installed langchain-0.3.15 langchain-openai-0.2.14 ...
```

**Note:** This may take a few minutes as it downloads all dependencies.

---

### Step 3: Set Up Environment Variables

Create a `.env` file in the project root directory to store your OpenAI API key:

#### For Mac/Linux:

```
# Create the .env file  
touch .env  
  
# Open it in a text editor (or use nano/vim)  
nano .env
```

#### For Windows:

```
# Create the .env file using notepad  
notepad .env
```

#### Add the following content:

```
OPENAI_API_KEY=sk-your-actual-api-key-here  
  
# Valkey connection (semantic cache backend)  
VALKEY_HOST=localhost  
VALKEY_PORT=6379
```

#### Important:

- Replace `sk-your-actual-api-key-here` with your actual OpenAI API key
  - `VALKEY_HOST` and `VALKEY_PORT` point to the local Valkey Docker container
  - Never share or commit this file to version control
  - The `.env` file should already be in `.gitignore`
- 

### Step 4: Verify Installation

Check that everything is installed correctly:

```
# Check Python version
python --version
# Should show: Python 3.8 or higher

# Check installed packages
pip list
# Should show all packages from requirements.txt

# Verify langchain installation
python -c "import langchain; print(langchain.__version__)"
# Should output: 0.3.15
```

---

## Step 5: Prepare Your Data

Make sure you have your PDF document in the correct location:

```
Symposium_Bot/
├── data/
│   └── symposium.pdf    ← Your PDF should be here
```

If you want to use a different PDF:

1. Place it in the **data/** folder
2. Update the path in [phase1\\_ingestion/ingest.py](#) (line 17)

---

## Running Commands with Virtual Environment

**IMPORTANT:** All Python commands and scripts must be run with the virtual environment activated.

**Always Activate First:**

**Mac/Linux:**

```
source .venv/bin/activate
```

**Windows:**

```
.venv\Scripts\activate
```

**Running Python Scripts:**

**Mac/Linux:**

```
# Run Phase 1 ingestion
python phase1_ingestion/ingest.py

# Run the embedding demo
python demo_embeddings.py

# Run the backend server
python3 -m uvicorn phase2_retrieval.backend:app --reload

# Run the frontend
python3 -m streamlit run phase2_retrieval/frontend.py
```

### Windows:

```
# Run Phase 1 ingestion
python phase1_ingestion/ingest.py

# Run the embedding demo
python demo_embeddings.py

# Run the backend server
python -m uvicorn phase2_retrieval.backend:app --reload

# Run the frontend
python -m streamlit run phase2_retrieval/frontend.py
```

---

## Deactivating the Virtual Environment

When you're done working on the project:

```
deactivate
```

This returns you to your system's default Python environment.

---

## Quick Start Summary

Here's the complete workflow for starting the project:

### Mac/Linux:

```
# 1. Activate virtual environment
source .venv/bin/activate

# 2. Run Phase 1 (only once)
```

```
python phase1_ingestion/ingest.py

# 3. (Optional) View embeddings demo
python demo_embeddings.py

# 4. Start backend (keep this running)
python3 -m uvicorn phase2_retrieval.backend:app --reload

# 5. In a new terminal, activate venv and start frontend
source .venv/bin/activate
python3 -m streamlit run phase2_retrieval/frontend.py
```

## Windows:

```
# 1. Activate virtual environment
.venv\Scripts\activate

# 2. Run Phase 1 (only once)
python phase1_ingestion/ingest.py

# 3. (Optional) View embeddings demo
python demo_embeddings.py

# 4. Start backend (keep this running)
python -m uvicorn phase2_retrieval.backend:app --reload

# 5. In a new terminal/command prompt, activate venv and start frontend
.venv\Scripts\activate
python -m streamlit run phase2_retrieval/frontend.py
```

---

## Troubleshooting Setup Issues

**Problem: "python: command not found" or "python3: command not found"**

**Solution:**

- On Mac/Linux, try using `python3` instead of `python`
- On Windows, make sure Python is added to your PATH during installation

---

**Problem: "pip: command not found"**

**Solution:**

```
# Try using python -m pip instead
python -m pip install -r requirements.txt
```



**Problem: "Permission denied" when creating virtual environment****Solution (Mac/Linux):**

```
# Use sudo for system-level permissions
sudo python3 -m venv .venv

# Or create in your home directory with proper permissions
chmod +x .venv/bin/activate
```

---

**Problem: Virtual environment activation doesn't work**

**Windows Solution:** If you get an execution policy error:

```
# Run PowerShell as Administrator
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# Then try activating again
.venv\Scripts\activate
```

---

**Problem: "No module named 'dotenv'" or similar import errors**

**Solution:** Make sure virtual environment is activated and reinstall:

```
# Check if venv is active (should see (.venv) in prompt)
# If not, activate it first

# Reinstall requirements
pip install -r requirements.txt
```

---

**Problem: `ModuleNotFoundError` when running uvicorn or streamlit via the bare command**

**Solution:** Use `python3 -m` prefix instead of the bare command:

```
python3 -m uvicorn phase2_retrieval.backend:app --reload
python3 -m streamlit run phase2_retrieval/frontend.py
```

This ensures the same Python interpreter that has your installed packages is used.

---

**Problem:** Backend fails to start with **ConnectionRefusedError** or **Connection refused** to port **6379**

**Cause:** The Valkey Docker container is not running.

**Solution:**

```
# Check if the container exists
docker ps -a | grep valkey-semantic-cache

# If stopped, start it
docker start valkey-semantic-cache

# If it doesn't exist, create it
docker run -d --name valkey-semantic-cache -p 6379:6379 valkey/valkey-
bundle:latest
```

---

**Problem:** **FT.CREATE failed** or **Unknown command** when starting the backend

**Cause:** You're running a Valkey image that doesn't include the search module.

**Solution:** Use **valkey/valkey-bundle:latest** (not **valkey/valkey:8.x**):

```
docker rm -f valkey-semantic-cache
docker run -d --name valkey-semantic-cache -p 6379:6379 valkey/valkey-
bundle:latest
```

---

**Problem:** Cache never hits even for identical questions

**Cause:** The **OPENAI\_API\_KEY** in **.env** may be missing or wrong, causing the embedding call to fail silently.

**Solution:** Verify the key is set in **.env** and restart the backend:

```
grep OPENAI_API_KEY .env
python3 -m uvicorn phase2_retrieval.backend:app --reload
```

---

## Phase 1: Data Ingestion

**File:** [phase1\\_ingestion/ingest.py](#)

**Purpose:** This script prepares your PDF document so the chatbot can search through it efficiently.

**When to run:** Only ONCE when you first set up the project (or when you change the PDF).

## Step-by-Step Breakdown

### Step 1: Load Environment Variables (Lines 6-14)

```
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()
```

#### What it does:

- Loads your OpenAI API key from the `.env` file
- This keeps your secret key safe (never put API keys directly in code!)

**Beginner tip:** The `.env` file looks like this:

```
OPENAI_API_KEY=sk-your-actual-key-here
```

---

### Step 2: Load the PDF (Lines 17-26)

```
from langchain_community.document_loaders import PyPDFLoader

pdf_path = "data/symposium.pdf"
loader = PyPDFLoader(pdf_path)
documents = loader.load()
```

#### What it does:

- Opens the PDF file at `data/symposium.pdf`
- Reads all the pages
- Creates a list of "documents" (one per page)

**Example:** If your PDF has 10 pages, `documents` will be a list with 10 items.

**What's in a document?** Each document has:

- `page_content`: The text from that page
  - `metadata`: Information like page number
- 

### Step 3: Split Text into Chunks (Lines 29-42)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,      # Each chunk will be ~1000 characters
    chunk_overlap=200,    # 200 character overlap between chunks
    length_function=len,
)

chunks = text_splitter.split_documents(documents)
```

### Why do we need chunks?

- AI models work better with smaller pieces of text
- It's easier to find relevant information in small chunks
- It's more cost-effective (you only process relevant chunks)

### What is `chunk_size=1000`?

- Each piece of text will be about 1000 characters long
- That's roughly 150-200 words or 1-2 paragraphs

### What is `chunk_overlap=200`?

- Each chunk overlaps with the next by 200 characters
- This ensures we don't split important information awkwardly

### Example:

```
Chunk 1: "The symposium will be held on March 15th... [1000 chars]"
Chunk 2: "...March 15th at the conference center... [1000 chars]"
        ↑ (This part overlaps with Chunk 1)
```

---

## Step 4: Create Embeddings (Lines 45-54)

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
```

**What are embeddings?** Embeddings are mathematical representations of text. They convert text into a list of numbers (a vector) that captures the meaning.

### Example:

```
Text: "The weather is sunny"
Embedding: [0.2, 0.5, -0.1, 0.8, ...] (1536 numbers)

Text: "It's a beautiful day"
Embedding: [0.19, 0.52, -0.09, 0.79, ...] (similar numbers!)
```

### Why embeddings?

- Sentences with similar meanings have similar embeddings
- This allows the computer to find relevant text based on meaning, not just keywords

### The Model: `text-embedding-3-small`

- This is OpenAI's embedding model
- "small" means it's fast and affordable
- It converts text into a vector of 1536 numbers

---

### Step 5: Store in Vector Store (Lines 57-67)

```
from langchain_community.vectorstores import FAISS

vector_store = FAISS.from_documents(
    documents=chunks,
    embedding=embeddings
)
```

**What is FAISS?** FAISS (Facebook AI Similarity Search) is a library that:

- Stores vectors (embeddings) efficiently
- Can quickly find similar vectors

### What happens here:

1. Takes each chunk of text
2. Converts it to an embedding (vector)
3. Stores all embeddings in the FAISS database

**Analogy:** Think of it as creating an index in a book. Instead of reading every page to find information, you look at the index to jump to the right page.

---

### Step 6: Save to Disk (Lines 70-79)

```
vector_store_path = "vector_store"
vector_store.save_local(vector_store_path)
```

## What it does:

- Saves the FAISS database to the [vector\\_store/](#) folder
- Creates two files:
  - `index.faiss`: The actual vector database
  - `index.pkl`: Metadata and mappings

## Why save to disk?

- So you don't have to recreate embeddings every time (saves time and money!)
  - Embeddings cost money because they call the OpenAI API
- 

## Summary of Phase 1

**Input:** PDF file **Output:** Vector database (FAISS index)

### Process:

1. PDF → Pages
2. Pages → Chunks (smaller pieces)
3. Chunks → Embeddings (numbers representing meaning)
4. Embeddings → Stored in FAISS database
5. Database → Saved to disk

**Cost:** Each chunk requires one API call to OpenAI for embeddings. A 10-page PDF might create 50 chunks = 50 API calls.

---

## Visualizing Embeddings (Optional Demo)

**File:** [demo\\_embeddings.py](#)

**Purpose:** After completing Phase 1, you can run this demonstration script to visualize and understand how the embedding process works.

**When to run:** AFTER Phase 1 is complete (when `vector_store` folder exists).

### How to run:

```
python demo_embeddings.py
```

## What This Demo Shows

This script provides a hands-on visualization of the embedding process:

### 1. Vector Store Statistics

- Total number of chunks created from your PDF
- Dimension of each vector (1536 numbers for `text-embedding-3-small`)

## 2. Sample Chunks and Their Embeddings

- Shows actual text chunks from your document
- Displays the corresponding vector embeddings (arrays of numbers)
- Reveals vector statistics (mean, standard deviation, min, max)

## 3. Vector Similarity

- Demonstrates how to calculate similarity between vectors
- Uses cosine similarity (ranges from -1 to 1)
- Helps understand how similar chunks are mathematically related

## 4. Semantic Search Demo

- Performs a test search query
- Shows which chunks are retrieved as most relevant
- Illustrates the retrieval process in action

### Example Output

When you run the demo, you'll see:

```
=====
=====
EMBEDDING DEMONSTRATION
=====
=====

[Step 1] Loading vector store...
✅ Vector store loaded successfully

[Step 2] Vector Store Statistics:
-----
-----
Total number of chunks: 42
Vector dimension: 1536

[Step 3] Sample Chunks and Their Embeddings:
-----
-----

=====
=====
EXAMPLE 1
=====
=====

📄 TEXT CHUNK 1:
-----
-----
Source: data/symposium.pdf
Page: 1
```

Content (first 300 characters):  
The symposium will be held on March 15th...

#### CORRESPONDING VECTOR EMBEDDING:

-----  
-----  
Vector dimension: 1536  
First 10 values: [ 0.0234 -0.0156 0.0892 0.0445 ...]  
Last 10 values: [-0.0123 0.0567 -0.0234 0.0891 ...]

#### VECTOR STATISTICS:

-----  
-----  
Mean: 0.000234  
Std Dev: 0.045678  
Min: -0.234567  
Max: 0.345678

## Key Learning Points

### Understanding Embeddings:

- Each chunk of text is converted into exactly 1536 numbers
- These numbers capture the semantic meaning of the text
- Similar meanings produce similar number patterns

### Vector Properties:

- Vectors are stored as NumPy arrays of floating-point numbers
- The numbers typically range between -1 and 1
- The dimension (1536) is determined by the embedding model

### Semantic Search:

- Your question is also converted to a vector
- FAISS compares your question vector to all chunk vectors
- The most similar vectors (chunks) are returned as results

**Why This Matters:** This visualization helps you understand that RAG is not magic—it's mathematics! Text gets converted to numbers, and the system finds chunks with similar numbers to answer your questions.

**Customization:** You can change the `NUM_SAMPLES` variable in the script (line 61) to see more or fewer examples.

---

## Phase 2: Backend API

**File:** [phase2\\_retrieval/backend.py](#)



**Purpose:** This is the "brain" of your application. It receives questions, searches the vector database, and generates answers.

**When to run:** Keep it running whenever you want to use the chatbot.

**How to run (from project root):**

```
python3 -m uvicorn phase2_retrieval.backend:app --reload
```

**Note:** Use `python3 -m uvicorn` (not a bare `uvicorn` command) to ensure the same Python installation that has your packages is used.

## Step-by-Step Breakdown

### Step 1: Setup and Imports (Lines 1-18)

```
import os
from contextlib import asynccontextmanager
from dotenv import load_dotenv
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
import valkey.asyncio as valkey_async
from betterdb_semantic_cache import SemanticCache, SemanticCacheOptions
from betterdb_semantic_cache.embed.openai import create_openai_embed
from betterdb_semantic_cache.types import CacheStoreOptions

load_dotenv()
```

#### What's happening:

- Imports all necessary libraries including `valkey.asyncio` (async Valkey client) and `betterdb_semantic_cache` (semantic cache library)
- `asynccontextmanager` is needed for FastAPI's lifespan pattern
- Loads environment variables (your API key and Valkey config)

---

### Step 2: Enable CORS (Lines 18-27)

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
```

```
    allow_headers=["*"],  
)
```

**What is CORS?** CORS stands for Cross-Origin Resource Sharing.

### Why do we need it?

- The frontend (Streamlit) runs on port 8501
- The backend (FastAPI) runs on port 8000
- Browsers block requests between different ports for security
- CORS middleware tells the browser "it's okay, allow these requests"

**Beginner tip:** `allow_origins=["*"]` means allow requests from anywhere. In production, you'd specify exact domains.

---

### Step 3: Load Vector Store (Lines 30-42)

```
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")  
vector_store = FAISS.load_local(  
    "vector_store",  
    embeddings,  
    allow_dangerous_deserialization=True  
)
```

### What it does:

- Loads the FAISS database we created in Phase 1
- Initializes the same embedding model (must match the one used in Phase 1!)

### Why `allow_dangerous_deserialization=True`?

- FAISS files use Python's pickle format
- Pickle can be dangerous if the file comes from an untrusted source
- Since we created the file ourselves, it's safe
- This parameter acknowledges that risk

**Important:** The embedding model here MUST be the same as in Phase 1!

---

### Step 3b: Initialize Semantic Cache via Lifespan (Lines 44-67)

```
semantic_cache: SemanticCache | None = None  
_valkey_client: valkey_async.Valkey | None = None  
  
@asynccontextmanager  
async def lifespan(app: FastAPI):  
    global semantic_cache, _valkey_client
```

```

_valkey_client = valkey_async.Valkey(
    host=os.getenv("VALKEY_HOST", "localhost"),
    port=int(os.getenv("VALKEY_PORT", 6379)),
)
semantic_cache = SemanticCache(SemanticCacheOptions(
    client=_valkey_client,
    embed_fn=create_openai_embed(model="text-embedding-3-small"),
    default_threshold=0.15,
    use_default_cost_table=True,
))
await semantic_cache.initialize()
yield
await semantic_cache.shutdown()
await _valkey_client.aclose()

app = FastAPI(lifespan=lifespan)

```

**What is the lifespan pattern?** FastAPI's **lifespan** context manager runs code before the server starts serving requests (**yield** marks the split point) and after it shuts down. This is the correct place to initialise resources that are async (like database connections) because the event loop is already running.

#### SemanticCacheOptions explained:

- **client** — the async Valkey connection
- **embed\_fn** — function to embed questions as vectors; uses the same model as the FAISS store so distances are comparable
- **default\_threshold=0.15** — cosine distance cutoff;  $\leq 0.15$  means a cache hit (questions  $\geq 85\%$  similar)
- **use\_default\_cost\_table=True** — loads BetterDB's built-in pricing table for 1,900+ models so cost savings are tracked automatically

`await semantic_cache.initialize()` runs **FT.CREATE** in Valkey to create the vector index. If the index already exists it's left unchanged.

#### Step 4: Initialize LLM (Lines 34)

```

llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0.7
)

```

**What is LLM?** LLM stands for Large Language Model (like ChatGPT).

**What is temperature?** Temperature controls how creative or focused the AI is:

- **0.0**: Very focused, deterministic (same question = same answer)
- **0.7**: Balanced (our setting)
- **1.0**: Very creative, random

### Analogy:

- Temperature 0 = Reading from a textbook
  - Temperature 0.7 = Having a conversation
  - Temperature 1 = Creative writing
- 

### Step 5: Define Data Models (Lines 55-66)

```
class ChatRequest(BaseModel):
    question: str

class ChatResponse(BaseModel):
    answer: str
    sources: list
    cache_hit: bool = False
    similarity: float | None = None
```

**What are these?** These define the structure of data sent to and from the API.

**ChatRequest:** What the frontend sends

```
{
  "question": "What is the symposium about?"
}
```

**ChatResponse:** What the backend returns

```
{
  "answer": "The symposium is about...",
  "sources": ["Page 1", "Page 3", "Page 5"],
  "cache_hit": false,
  "similarity": null
}
```

On a cache hit, **sources** is an empty list (we skipped the FAISS search), **cache\_hit** is **true**, and **similarity** holds the computed similarity score (0.0–1.0, where 1.0 is identical).

### Why use Pydantic models?

- Automatic validation (ensures data is correct)
  - Type checking
  - Automatic API documentation
- 

### Step 6: The Chat Endpoint

This is the core of the application. Let's break it down into sub-steps:

```
@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
```

### What is `@app.post("/chat")`?

- This creates an API endpoint at `http://localhost:8000/chat`
- It accepts POST requests (requests that send data)
- It is now `async` because the semantic cache operations are async (Valkey I/O)

---

#### Step 6.0: Semantic Cache Check (runs before anything else)

```
cache_result = await semantic_cache.check(request.question)

if cache_result.hit:
    similarity_score = round(1 - (cache_result.similarity or 0), 4)
    print(f"[CACHE HIT] similarity={similarity_score:.4f} \n"
          {request.question}\n")
    return ChatResponse(answer=cache_result.response, sources=[],
                        cache_hit=True, similarity=similarity_score)
```

On a cache hit, the function returns immediately — skipping FAISS search and the LLM call entirely. `cache_result.similarity` is the raw cosine *distance* (0=identical), so `1 - distance` is converted to a similarity score before returning it.

---

#### Step 6.1: Semantic Search (only runs on a cache miss)

```
relevant_docs = vector_store.similarity_search(
    request.question,
    k=3
)

context = "\n\n".join([doc.page_content for doc in relevant_docs])

sources = [
    f"Page {doc.metadata.get('page', 'Unknown')}"
    for doc in relevant_docs
]
```

### What is similarity search?

1. Takes your question: "What is the date of the symposium?"
2. Converts it to an embedding (vector of numbers)

3. Finds the 3 chunks with the most similar embeddings
4. Returns those chunks

### How does it work?

```
Question Embedding:    [0.5, 0.2, 0.8, ...]
Chunk 1 Embedding:    [0.49, 0.21, 0.79, ...] ← Very similar!
Chunk 2 Embedding:    [0.1, 0.9, 0.3, ...]   ← Not similar
Chunk 3 Embedding:    [0.51, 0.19, 0.81, ...] ← Very similar!
```

### What is **k=3**?

- Return the top 3 most relevant chunks
- More chunks = more context but also more noise
- 3 is a good balance

### Building the context:

```
context = "\n\n".join([doc.page_content for doc in relevant_docs])
```

This combines the 3 chunks into one big text with blank lines between them.

### Example result:

```
Chunk 1 text goes here...

Chunk 2 text goes here...

Chunk 3 text goes here...
```

**Extracting sources:** This creates a list like: ["Page 5", "Page 12", "Page 3"]

---

### Step 6.2: Build Prompt (Lines 97-113)

```
prompt_template = ChatPromptTemplate.from_messages([
    ("system", """You are a helpful assistant that answers questions based
on the provided context.
Use the context below to answer the question. If you cannot find the answer
in the context, say so.

Context:
{context}"""),
    ("user", "{question}")
])
```

```
prompt = prompt_template.format_messages(  
    context=context,  
    question=request.question  
)
```

**What is a prompt?** A prompt is the instruction you give to the AI. It has two parts:

1. **System message:** Sets the AI's behavior and provides context
2. **User message:** The actual question

**Example of the final prompt:**

```
System: You are a helpful assistant...
```

```
Context:
```

```
[Text from Chunk 1]
```

```
[Text from Chunk 2]
```

```
[Text from Chunk 3]
```

```
User: What is the date of the symposium?
```

**Why include context?** Without context, GPT-3.5 doesn't know anything about your specific document. The context gives it the information it needs to answer.

---

### Step 6.3: Generate Response (Lines 116-121)

```
response = llm.invoke(prompt)  
answer = response.content
```

**What happens here:**

1. Sends the prompt to OpenAI's GPT-3.5-turbo model
2. The model reads the context and question
3. Generates an answer based on the provided chunks
4. Returns the answer

**Example:**

```
Prompt: [Context about symposium] + "What is the date?"
```

```
Response: "According to the document, the symposium will be held on March  
15th, 2024."
```

---

### Step 6.4: Store in Cache and Return Response

```
input_tokens = (response.usage_metadata or {}).get("input_tokens", 0)
output_tokens = (response.usage_metadata or {}).get("output_tokens", 0)
await semantic_cache.store(
    request.question,
    answer,
    CacheStoreOptions(model="gpt-3.5-turbo", input_tokens=input_tokens,
output_tokens=output_tokens),
)

return ChatResponse(answer=answer, sources=sources, cache_hit=False)
```

**Cache store:** Saves the question embedding + answer in Valkey. Token counts from `response.usage_metadata` are passed so BetterDB can calculate the exact dollar cost saved on every future cache hit for this question.

**What it returns:**

```
{
  "answer": "The symposium will be held on March 15th, 2024.",
  "sources": ["Page 5", "Page 12", "Page 3"],
  "cache_hit": false,
  "similarity": null
}
```

This gets sent back to the frontend.

---

## Step 7: Health Check Endpoint (Lines 134-140)

```
@app.get("/")
def root():
    return {"status": "Backend is running", "message": "Use /chat endpoint
to ask questions"}
```

**What is this?** A simple endpoint to check if the server is running.

**How to test:** Open your browser and go to <http://localhost:8000>

You'll see:

```
{
  "status": "Backend is running",
  "message": "Use /chat endpoint to ask questions"
}
```



---

## Summary of Backend API

**Purpose:** Handle chat requests with semantic caching, falling back to RAG + LLM generation on a miss.

### Endpoints:

- **GET /** - Health check
- **POST /chat** - Main chat endpoint (cache-aware)
- **GET /stats** - Returns cumulative cache statistics (hits, misses, hit rate, cost saved)

### Flow:

1. Receive question
  2. **Check semantic cache** — return immediately on a hit
  3. (miss only) Search vector database for relevant chunks
  4. Build prompt with context
  5. Send to OpenAI for answer generation
  6. Return answer and sources
- 

## Phase 2: Frontend UI

**File:** [phase2\\_retrieval/frontend.py](#)

**Purpose:** Provides a web-based chat interface for users to interact with the bot.

**When to run:** Keep it running whenever you want to use the chatbot.

### How to run:

```
streamlit run phase2_retrieval/frontend.py
```

## Step-by-Step Breakdown

### Step 1: Page Configuration (Lines 10-17)

```
import streamlit as st

st.set_page_config(
    page_title="Symposium RAG Bot",
    page_icon="🤖",
    layout="centered"
)
```

**What is Streamlit?** Streamlit is a Python library that makes it easy to create web interfaces without knowing HTML, CSS, or JavaScript.

## What does this do?

- Sets the browser tab title to "Symposium RAG Bot"
  - Sets the browser tab icon to a robot emoji
  - Centers the content on the page
- 

## Step 2: Backend Configuration (Lines 20-24)

```
BACKEND_URL = "http://localhost:8000/chat"
```

**What is this?** The URL where the frontend will send questions.

### Breakdown:

- `http://` - Protocol
  - `localhost` - Your own computer
  - `8000` - Port where the backend is running
  - `/chat` - The endpoint we defined in the backend
- 

## Step 3: Initialize Chat History (Lines 27-32)

```
if "messages" not in st.session_state:  
    st.session_state.messages = []
```

**What is `st.session_state`?** Streamlit reruns your entire script every time something changes. `session_state` is a way to store data that persists between reruns.

### What is this doing?

- Checks if `messages` exists in session state
- If not, creates an empty list
- This stores the entire chat history

### Example of chat history:

```
[  
    {"role": "user", "content": "What is the symposium about?"},  
    {"role": "assistant", "content": "The symposium is about...", "sources":  
["Page 1"]},  
    {"role": "user", "content": "When is it?"},  
    {"role": "assistant", "content": "It's on March 15th.", "sources": ["Page  
5"]}  
]
```

---

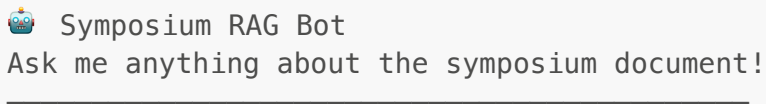
## Step 4: UI Header (Lines 35-41)

```
st.title("🤖 Symposium RAG Bot")
st.markdown("Ask me anything about the symposium document!")
st.divider()
```

### What does this create?

- A large title at the top of the page
- A subtitle with instructions
- A horizontal line to separate the header

### Visual result:



## Step 5: Display Chat History (Lines 44-56)

```
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

    if message["role"] == "assistant" and "sources" in message:
        with st.expander("📖 Sources"):
            for source in message["sources"]:
                st.caption(source)
```

**What does this do?** Loops through all previous messages and displays them.

### `st.chat_message(message["role"]):`

- Creates a chat bubble
- `role` can be "user" or "assistant"
- Automatically adds the appropriate icon and styling

### `st.expander("📖 Sources"):`

- Creates a collapsible section
- Starts collapsed
- User can click to expand and see sources

### Visual result:

 User

What is the symposium about?

 Assistant

The symposium is about artificial intelligence...

 Sources ▼

Page 1

Page 5

---

## Step 6: User Input Handling (Lines 59-73)

```
if user_question := st.chat_input("Type your question here..."):

    with st.chat_message("user"):
        st.markdown(user_question)

    st.session_state.messages.append({
        "role": "user",
        "content": user_question
    })
```

**What is `:=` (walrus operator)?** It's a Python operator that assigns AND checks in one line.

This:

```
if user_question := st.chat_input("..."):
```

Is equivalent to:

```
user_question = st.chat_input("...")
if user_question:
```

### `st.chat_input()`:

- Creates a text input box at the bottom of the page
- Returns `None` until the user types something and presses Enter
- Returns the user's text when they submit

### What happens when user submits:

1. Store the question in `user_question`
2. Display it in a user chat bubble
3. Add it to chat history

---

## Step 7: Call Backend API (Lines 76-91)

```
with st.chat_message("assistant"):
    with st.spinner("Thinking..."):

        response = requests.post(
            BACKEND_URL,
            json={"question": user_question}
        )

        result = response.json()
        answer = result["answer"]
        sources = result.get("sources", [])
        cache_hit = result.get("cache_hit", False)
        similarity = result.get("similarity")
```

### Breaking it down:

#### `st.spinner("Thinking..."):`

- Shows a loading animation while waiting
- Displays "Thinking..." text

#### `requests.post():`

- Sends an HTTP POST request to the backend
- `json={"question": user_question}` sends the question as JSON

### Example request:

```
POST http://localhost:8000/chat
{
  "question": "What is the symposium about?"
}
```

#### `response.json():`

- Converts the response to a Python dictionary
- Now also extracts `cache_hit` (bool) and `similarity` (float or None)

### Example response (cache miss):

```
{
  "answer": "The symposium is about AI...",
  "sources": ["Page 1", "Page 3"],
  "cache_hit": false,
```

```
"similarity": null
}
```

### Example response (cache hit):

```
{
  "answer": "The symposium is about AI...",
  "sources": [],
  "cache_hit": true,
  "similarity": 0.9700
}
```

## Step 8: Display Assistant Response

```
st.markdown(answer)

if cache_hit and similarity is not None:
    st.caption(f"< Cached · similarity {similarity:.4f}")
elif sources:
    with st.expander("📖 Sources"):
        for source in sources:
            st.caption(source)
```

### What it does:

- Displays the answer in the assistant chat bubble
- On a **cache hit**: shows a < Cached · similarity X.XXXX caption instead of sources (no FAISS search was done)
- On a **cache miss**: shows the collapsible Sources section as before

### Visual result (cache hit):

```
🤖 Assistant
The symposium focuses on artificial intelligence and machine learning...
< Cached · similarity 0.9700
```

## Step 9: Save to Chat History

```
st.session_state.messages.append({
    "role": "assistant",
    "content": answer,
    "sources": sources,
```

```
"cache_hit": cache_hit,  
"similarity": similarity,  
})
```

**What it does:** Adds the assistant's response to chat history so it persists if the page refreshes. The `cache_hit` and `similarity` values are stored so the badge is shown correctly when the history is replayed.

---

## Summary of Frontend

**Purpose:** Provide a user-friendly chat interface

**Features:**

- Chat input box
- Chat history (persists during session)
- Loading indicator
- Source citations (on cache miss)
- ⚡ **Cached** badge with similarity score (on cache hit)

**Flow:**

1. User types question
  2. Display user message
  3. Send request to backend
  4. Show "Thinking..." spinner
  5. Receive answer, `cache_hit`, and `similarity` from backend
  6. Display answer with cache badge or sources depending on hit/miss
  7. Save to chat history (including cache metadata)
- 
- 

## Phase 3: Semantic Caching with BetterDB

This phase adds a semantic caching layer between the user's question and the LLM, powered by [BetterDB](#) and [Valkey](#).

### What Problem Does It Solve?

Every question sent to GPT-3.5-turbo costs money and takes time (~1-2 seconds). When multiple users ask similar questions — or the same user rephrases a question — the LLM is called again unnecessarily.

**Standard caching** only catches exact matches:

- "What is the event date?" → MISS (first time)
- "What is the event date?" → HIT (exact repeat)
- "When does the event take place?" → MISS (missed — different words, same meaning)

**Semantic caching** catches paraphrases too:

- "What is the event date?" → MISS (first time, stored in cache)
- "When does the event take place?" → HIT (similarity score: 0.97)

---

## Infrastructure: Valkey

Valkey is the open-source key-value store used as the storage backend for the cache. It needs the `valkey-search` module to support vector indexing (the `FT.*` commands).

### Run Valkey locally via Docker:

```
docker run -d \  
  --name valkey-semantic-cache \  
  -p 6379:6379 \  
  valkey/valkey-bundle:latest
```

`valkey/valkey-bundle` is the official Valkey Docker image that ships with all modules pre-loaded, including `valkey-search 1.2.0`.

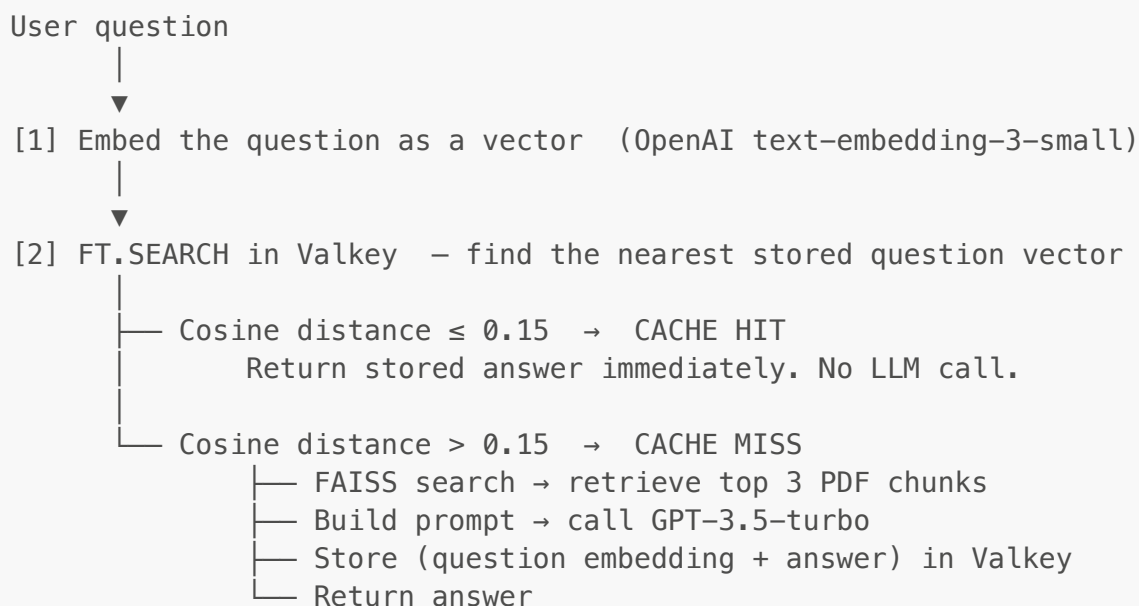
### Verify vector search is available:

```
docker exec -it valkey-semantic-cache valkey-cli FT._LIST  
# Expected: (empty array) - engine ready, no indexes yet
```

---

## How the Cache Works

When a question arrives at the backend:





**Threshold explained:** Cosine distance ranges from 0 (identical) to 1 (completely different). A threshold of **0.15** means "accept as a cache hit if the questions are at least 85% similar." This is tight enough to avoid wrong answers but loose enough to catch natural paraphrases.

---

## Changes Made to the Project

### .env

Added two new variables so the Valkey host and port are configurable without touching code:

```
VALKEY_HOST=localhost
VALKEY_PORT=6379
```

### requirements.txt

Added:

```
betterdb-semantic-cache
```

## backend.py — Key Changes

### 1. Lifespan context manager (startup/shutdown)

The **SemanticCache** is async and must be initialised before the server starts taking requests. FastAPI's **lifespan** is the correct place for this:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    _valkey_client = valkey_async.Valkey(
        host=os.getenv("VALKEY_HOST", "localhost"),
        port=int(os.getenv("VALKEY_PORT", 6379)),
    )
    semantic_cache = SemanticCache(SemanticCacheOptions(
        client=_valkey_client,
        embed_fn=create_openai_embed(model="text-embedding-3-small"),
        default_threshold=0.15,
        use_default_cost_table=True,
    ))
    await semantic_cache.initialize() # Creates the FT index in Valkey
    yield
    await semantic_cache.shutdown()
    await _valkey_client.aclose()
```

**use\_default\_cost\_table=True** loads BetterDB's built-in pricing table (1,900+ models) so cost savings are tracked automatically — no configuration needed.

## 2. Cache check at the top of `/chat`

```
@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    cache_result = await semantic_cache.check(request.question)

    if cache_result.hit:
        print(f"[CACHE HIT] similarity={...} \"{request.question}\"")
        return ChatResponse(answer=cache_result.response, ...)
```

On a hit, the function returns immediately — the FAISS search and LLM call are skipped entirely.

## 3. Cache store after every LLM call

```
await semantic_cache.store(
    request.question,
    answer,
    CacheStoreOptions(
        model="gpt-3.5-turbo",
        input_tokens=input_tokens,
        output_tokens=output_tokens,
    ),
)
```

Token counts are extracted from the LangChain response and passed to `store()` so BetterDB can calculate the exact dollar cost saved on future hits.

## 4. Terminal stats after every request

```
async def _print_stats() -> None:
    s = await semantic_cache.stats()
    print(
        f"[STATS] total={s.total} | hits={s.hits} | misses={s.misses} "
        f"| hit_rate={s.hit_rate:.1%} | saved=${s.cost_saved_micros / "
        f"1_000_000:.4f}"
    )
```

This gives a running view in the terminal of how the cache is performing.

## 5. `/stats` endpoint

```
@app.get("/stats")
async def stats():
    s = await semantic_cache.stats()
    return {
        "hits": s.hits,
```

```

    "misses": s.misses,
    "total": s.total,
    "hit_rate": f"{s.hit_rate:.1%}",
    "cost_saved_usd": f"${s.cost_saved_micros / 1_000_000:.4f}",
}

```

Accessible at <http://localhost:8000/stats>.

### frontend.py — Key Changes

The `ChatResponse` model now includes `cache_hit` (bool) and `similarity` (float). The frontend uses these to display a badge on cached responses:

```

if cache_hit:
    st.caption(f"< Cached · similarity {similarity:.4f}")

```

This badge appears both on the live response and when the chat history is replayed.

---

### Terminal Output Example

```

[CACHE MISS] "What is the event date?"
[STATS]      total=1 | hits=0 | misses=1 | hit_rate=0.0% | saved=$0.0000

[CACHE HIT]  similarity=0.9700 "When does the event take place?"
[STATS]      total=2 | hits=1 | misses=1 | hit_rate=50.0% | saved=$0.0008

```

---

### Troubleshooting — Semantic Cache

**Problem:** `ValkeyCommandError: FT.CREATE failed`

**Cause:** Valkey is running but the search module isn't loaded. **Solution:** Use `valkey/valkey-bundle:latest` (not `valkey/valkey:8.x`).

**Problem:** `Connection refused on port 6379`

**Solution:** Start the Docker container:

```
docker start valkey-semantic-cache
```

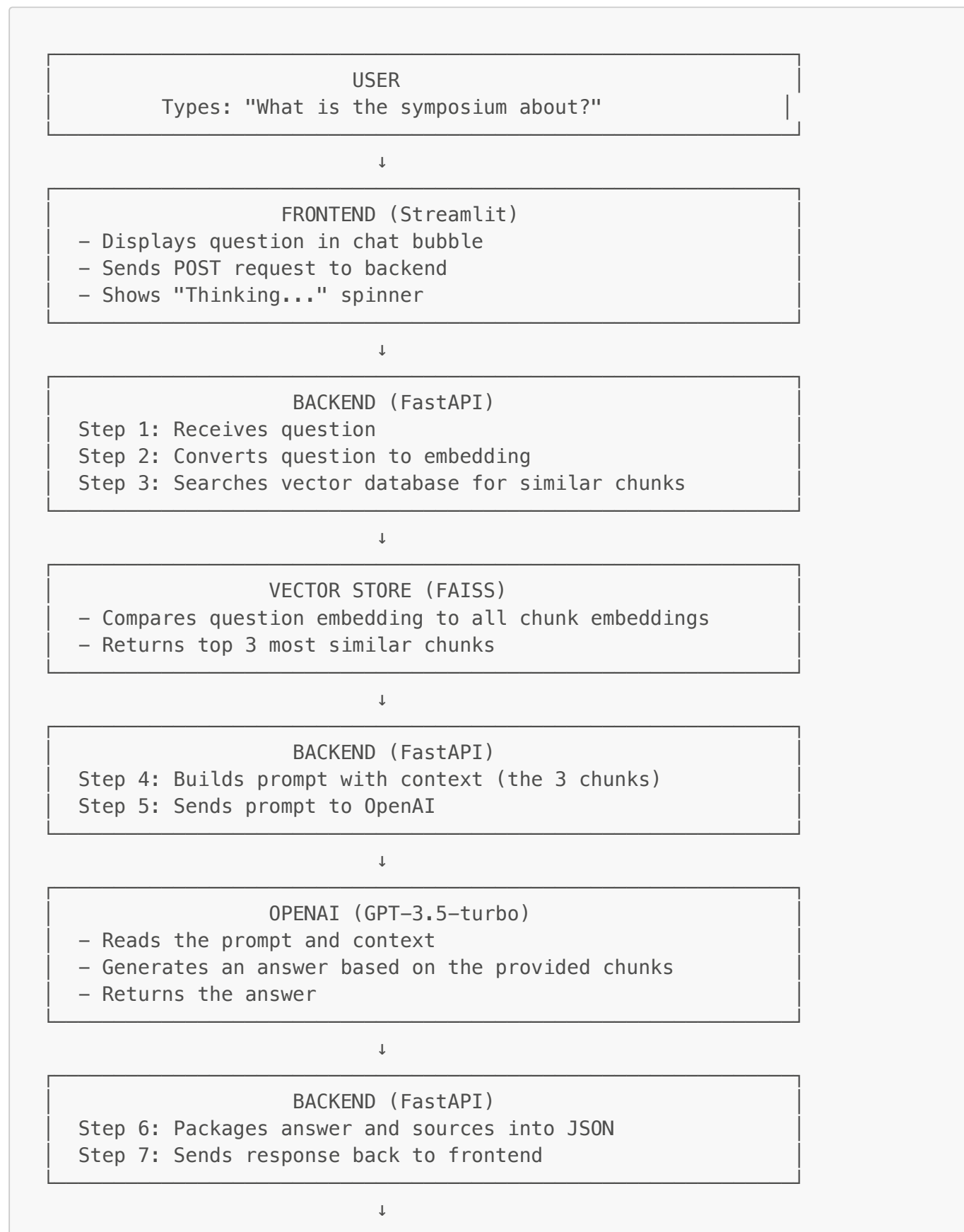
**Problem:** Cache never hits even for identical questions

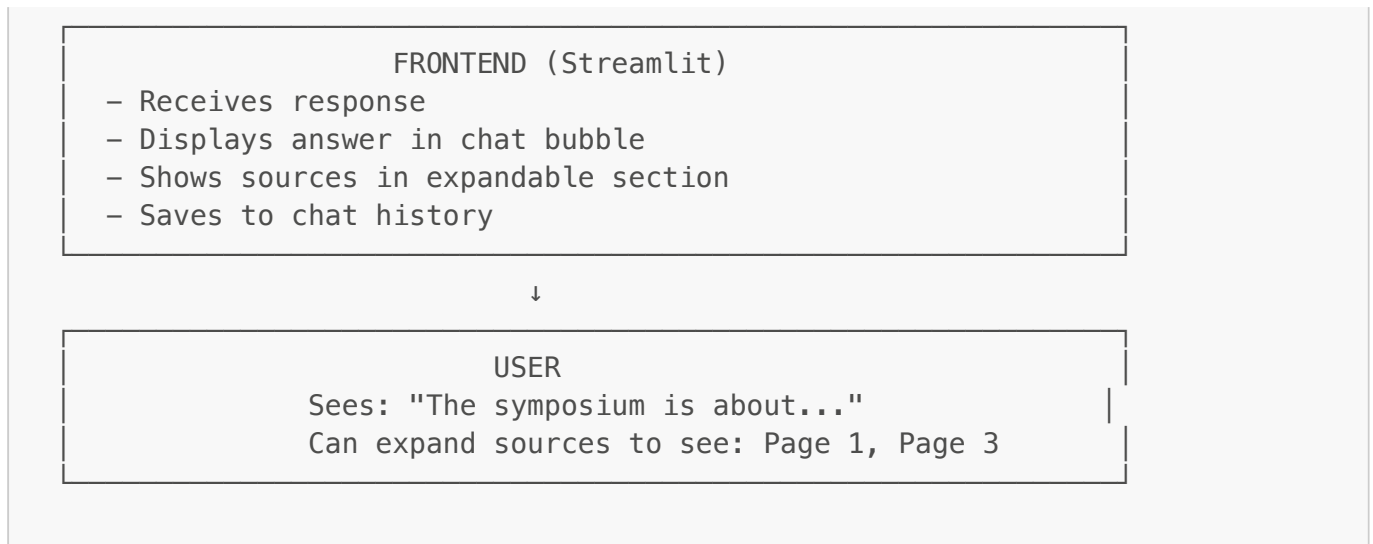
**Cause:** The `OPENAI_API_KEY` in `.env` may be wrong, causing the embedding call to fail silently.

**Solution:** Verify the key is set and valid, then restart the backend.

## How Everything Works Together

### The Complete Flow





---

## Key Concepts Explained

### 1. What are Embeddings?

**Simple explanation:** Embeddings turn text into numbers so computers can understand meaning.

**Example:**

```
"dog"      → [0.5, 0.2, 0.9, ...]
"puppy"    → [0.52, 0.19, 0.88, ...] ← Similar numbers!
"car"      → [0.1, 0.8, 0.3, ...]    ← Different numbers
```

**Why useful?** The computer can calculate: "dog" and "puppy" are similar (even though the words are different).

---

### 2. What is Vector Similarity Search?

**The problem:** You have 100 chunks of text. How do you find the most relevant ones?

**Traditional approach:** Search for keywords. Problem: Misses synonyms and context.

**Vector approach:**

1. Convert question to embedding: **[0.5, 0.2, 0.9, ...]**
2. Compare with all chunk embeddings
3. Find chunks with most similar numbers
4. Return those chunks

**Similarity calculation:** Uses math (cosine similarity or Euclidean distance) to measure how "close" two vectors are.

---

### 3. What is the Temperature Parameter?

Temperature controls randomness in AI responses:

| Temperature | Behavior               | Example Use Case             |
|-------------|------------------------|------------------------------|
| 0.0         | Focused, deterministic | Math problems, factual Q&A   |
| 0.3-0.7     | Balanced (default)     | General chatbots             |
| 0.8-1.0     | Creative, varied       | Story writing, brainstorming |

**Example with temperature 0:**

- Question: "What is 2+2?"
- Always answers: "4"

**Example with temperature 1:**

- Question: "What is 2+2?"
- Might answer: "4", "Four", "Two plus two equals four", etc.

---

## 4. What is Chunking and Why Do We Need It?

**The problem:**

- Your PDF might be 100 pages
- AI models have token limits (can't process entire document at once)
- Processing entire document is expensive

**The solution:** Break into smaller, manageable pieces (chunks).

**Good chunk size:**

- Too small (100 chars): Not enough context
- Too large (5000 chars): Too much irrelevant info
- Just right (1000 chars): ~1-2 paragraphs

**Chunk overlap:** Prevents breaking sentences or ideas awkwardly.

---

## 5. What is the Difference Between Retrieval and Generation?

**Retrieval:**

- Finding relevant information
- Done by FAISS vector search
- Fast and cheap

**Generation:**

- Creating new text based on retrieved information
- Done by GPT-3.5-turbo
- Slower and more expensive

## Why separate?

- Don't send entire document to GPT (expensive!)
  - First retrieve only relevant parts (fast)
  - Then generate answer from relevant parts (efficient)
- 

## 6. What is an API?

### API = Application Programming Interface

**Simple explanation:** A way for programs to talk to each other.

#### Our example:

- Frontend: "Hey backend, answer this question!"
- Backend: "Sure! Here's the answer and sources."

**How they communicate:** Using HTTP requests (like your browser does with websites).

---

## 7. Why Do We Need Both Frontend and Backend?

### Frontend (Streamlit):

- What the user sees and interacts with
- Handles display, input, chat history
- Simple, user-friendly

### Backend (FastAPI):

- Does the heavy work
- Vector search, AI processing
- Can be used by multiple frontends
- Can scale independently

#### Analogy:

- Frontend = Restaurant (where customers interact)
  - Backend = Kitchen (where food is prepared)
- 

## 8. What is CORS and Why Do We Need It?

### CORS = Cross-Origin Resource Sharing

#### The problem:

- Frontend runs on `http://localhost:8501`
- Backend runs on `http://localhost:8000`
- Different ports = different "origins"
- Browsers block requests between different origins (security feature)

**The solution:** Backend says "I allow requests from other origins" using CORS middleware.

---

## 9. How Does Chat History Work?

**Streamlit's behavior:** Every time you interact with the page, Streamlit reruns the entire script.

**The problem:** Chat history would disappear on every rerun!

**The solution:** `st.session_state` stores data that persists between reruns.

**How it works:**

```
# First run: Creates empty list
if "messages" not in st.session_state:
    st.session_state.messages = []

# Add message
st.session_state.messages.append({"role": "user", "content": "Hi"})

# Next rerun: List still exists with the message!
```

---

## 10. What is Pydantic and Why Use It?

**Pydantic:** A library for data validation using Python type hints.

**Without Pydantic:**

```
def chat(request):
    question = request["question"] # Might not exist! Might be wrong type!
```

**With Pydantic:**

```
class ChatRequest(BaseModel):
    question: str

def chat(request: ChatRequest):
    question = request.question # Guaranteed to exist and be a string!
```

**Benefits:**

- Automatic validation
  - Better error messages
  - Type checking
  - Auto-generated API docs
-



## Common Questions

Q: Why OpenAI? Can I use other models?

A: Yes! You can replace OpenAI with:

- Other APIs: Anthropic (Claude), Google (Gemini)
- Local models: Ollama, LM Studio
- Open source: Llama, Mistral

Just replace the `ChatOpenAI` and `OpenAIEmbeddings` classes.

---

Q: How much does this cost?

A: Costs depend on usage:

**One-time (Phase 1):**

- Embeddings: ~\$0.01 per 100 pages

**Per question (Phase 2):**

- Embedding query: ~\$0.00002
- GPT-3.5-turbo: ~\$0.002 per question

**Example:** 1000 questions ≈ \$2-3

---

Q: Can this work with multiple PDFs?

A: Yes! Modify Phase 1 to loop through multiple PDFs:

```
pdf_files = ["doc1.pdf", "doc2.pdf", "doc3.pdf"]
all_chunks = []

for pdf in pdf_files:
    documents = PyPDFLoader(pdf).load()
    chunks = text_splitter.split_documents(documents)
    all_chunks.extend(chunks)

vector_store = FAISS.from_documents(all_chunks, embeddings)
```

---

Q: How accurate are the answers?

**Factors affecting accuracy:**

1. **Quality of chunks:** Better chunking = better retrieval
2. **Number of retrieved chunks (k):** More chunks = more context
3. **Prompt design:** Better prompts = better answers

#### 4. **Model choice:** GPT-4 > GPT-3.5 in accuracy

---

Q: What if the answer isn't in the PDF?

The system prompt tells the AI:

```
"If you cannot find the answer in the context, say so."
```

So it should respond: "I cannot find that information in the provided document."

---

Q: Can I deploy this?

**Yes!** Common deployment options:

**Backend:**

- Railway, Render, Fly.io, AWS

**Frontend:**

- Streamlit Community Cloud (free!)
- Heroku, Vercel

**Vector Store:**

- Keep using FAISS (file-based)
  - Or upgrade to: Pinecone, Weaviate, Qdrant (cloud vector databases)
- 

## Next Steps for Learning

### Beginner Level

- ☒ Understand how each file works
- ☒ Modify the chunk size and see how it affects results
- ☒ Try different questions
- ☒ Change the temperature and observe differences

### Intermediate Level

- Add error handling (try/except blocks)
- Support multiple PDFs
- Add logging to track what's happening
- Improve the UI with more Streamlit components

### Advanced Level

- Implement conversation memory (remember previous questions)
- Add authentication
- Use a cloud vector database

- Implement caching to reduce API calls
  - Add metrics and monitoring
- 

## Troubleshooting

Problem: "ModuleNotFoundError"

**Solution:** Install dependencies

```
pip install -r requirements.txt
```

---

Problem: "OpenAI API Error"

**Solution:** Check your [.env](#) file has the correct API key

---

Problem: "Vector store not found"

**Solution:** Run Phase 1 first

```
python phase1_ingestion/ingest.py
```

---

Problem: "Connection refused" when frontend tries to reach backend

**Solution:** Make sure backend is running

```
.venv/bin/uvicorn phase2_retrieval.backend:app --reload
```

---

Problem: Answers are irrelevant

**Solutions:**

- Increase **k** (retrieve more chunks)
  - Adjust chunk size
  - Improve chunking strategy
  - Use a better embedding model
- 

## Conclusion

This RAG bot demonstrates the fundamental concepts of retrieval-augmented generation:

1. **Preparation:** Convert documents into searchable embeddings
2. **Retrieval:** Find relevant information based on queries
3. **Generation:** Use AI to synthesize answers from retrieved context

The beauty of this architecture is:

- **Scalable:** Can work with large documents
- **Accurate:** Answers are grounded in your specific documents
- **Cost-effective:** Only processes relevant chunks
- **Extensible:** Easy to add features and improvements

Now you understand not just how to use this RAG bot, but how every piece works and why it's designed this way!

Happy learning! 🚀